Take Home Activity 2 - V1

TEAM NUMBER 24

Adapter Pattern : Unifying Aircraft Types

WHERE Is It Needed

The Adapter Pattern is needed in the **aircraft/plane subsystem** and the classes that consume it:

- **`flight.reservation.plane.PassengerPlane`** — has public fields `model`, `passengerCapacity`, `crewCapacity`
- **`flight.reservation.plane.Helicopter`** — has methods `getModel()`, `getPassengerCapacity()`; crew capacity is not stored
- **`flight.reservation.plane.PassengerDrone`** — has only a `model` field; capacity and crew info are not stored at all

These three unrelated classes are used by:

- **`flight.reservation.flight.Flight`** (lines 15, 31–43) — stores the aircraft as `Object` and uses `instanceof` chains to extract the model string
- **`flight.reservation.flight.ScheduledFlight`** (lines 32–40, 48–56) — uses `instanceof` chains to determine passenger capacity and crew capacity
- **`Runner.java`** (line 22) — stores all aircraft in `List<Object>` because there is no common type

DESCRIPTION of the Pattern

The **Adapter Pattern** is a structural design pattern that allows objects with incompatible interfaces to work together. It wraps an existing class with a new interface so that it becomes compatible with what the client expects.

In this codebase, `PassengerPlane`, `Helicopter`, and `PassengerDrone` all represent aircraft but expose their data in completely different ways (public fields vs. getter methods vs. no accessor at all for some properties). The Adapter Pattern introduces a common `Aircraft` interface:

```
public interface Aircraft {
    String getModel();
    int getPassengerCapacity();
    int getCrewCapacity();
}
```

Each existing aircraft class is either:

- **Modified to implement** `Aircraft` directly (Object Adapter / simple refactor), or
- **Wrapped in an adapter class** (e.g., `PassengerDroneAdapter implements Aircraft`) that translates its specific API into the common interface — this is the classic Adapter approach, preferred when you cannot modify the original classes.

After applying the pattern:

- `Flight` stores `Aircraft` instead of `Object`
- All `instanceof` chains are replaced with polymorphic method calls like `aircraft.getModel()`
- `ScheduledFlight.getCapacity()` simply calls `aircraft.getPassengerCapacity()` — no branching needed

WHY Is It Needed / WHAT Is the Issue Without It

Current Problems

1. **`Object` as the aircraft type** — `Flight.java` declares `protected Object aircraft` because there is no common type. This discards all type safety; any object can be passed as an aircraft.

2. **Repeated `instanceof` chains** — The same pattern of checking `instanceof PassengerPlane`, `instanceof Helicopter`, `instanceof PassengerDrone` appears in **three separate methods** across two classes:

   In `Flight.isAircraftValid()`:

   ```
   if (this.aircraft instanceof PassengerPlane) {
       model = ((PassengerPlane) this.aircraft).model;
   } else if (this.aircraft instanceof Helicopter) {
       model = ((Helicopter) this.aircraft).getModel();
   } else if (this.aircraft instanceof PassengerDrone) {
       model = "HypaHype";  // hardcoded!
   }
   ```

   In `ScheduledFlight.getCapacity()`:

   ```
   if (this.aircraft instanceof PassengerPlane) {
       return ((PassengerPlane) this.aircraft).passengerCapacity;
   }
   if (this.aircraft instanceof Helicopter) {
       return ((Helicopter) this.aircraft).getPassengerCapacity();
   }
   if (this.aircraft instanceof PassengerDrone) {
       return 4;  // hardcoded!
   }
   ```

   In `ScheduledFlight.getCrewMemberCapacity()`:

   ```
   if (this.aircraft instanceof PassengerPlane) {
       return ((PassengerPlane) this.aircraft).crewCapacity;
   }
   if (this.aircraft instanceof Helicopter) {
       return 2;  // hardcoded!
   }
   if (this.aircraft instanceof PassengerDrone) {
       return 0;  // hardcoded!
   }
   ```

3. **Hardcoded values scattered across consumers** — The `PassengerDrone` capacity of `4`, `Helicopter` crew capacity of `2`, and drone crew capacity of `0` are hardcoded inside `ScheduledFlight`, not inside the aircraft classes where they belong. This means the aircraft's own properties are defined somewhere else entirely.

4. **Adding a new aircraft type is fragile** — If you introduce a new type (e.g., `Glider`), you must find and update **every single `instanceof` chain** in `Flight` and `ScheduledFlight`. Missing even one results in an unhandled case and a runtime exception.
5. **Inconsistent APIs** — `PassengerPlane` uses public fields (`model`, `passengerCapacity`), `Helicopter` uses getter methods (`getModel()`), and `PassengerDrone` doesn't expose capacity at all. There is no uniform way to interact with aircraft.

Benefits of Applying the Adapter Pattern

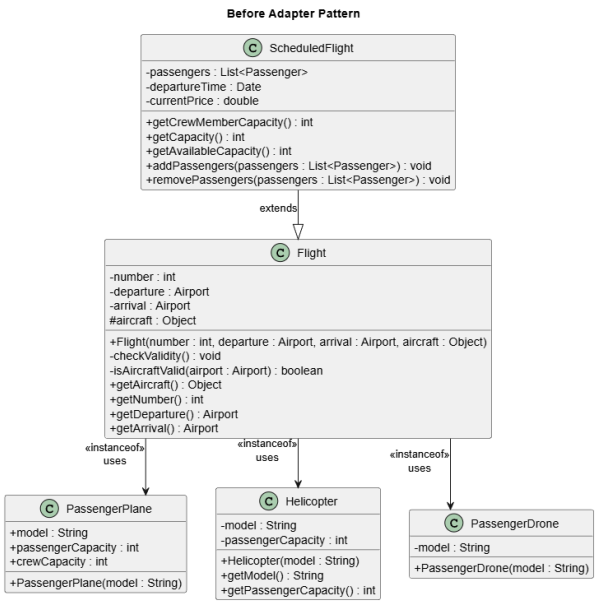
- **Eliminates all `instanceof` checks** — Polymorphic calls (`aircraft.getModel()`) replace every `instanceof` branch in `Flight` and `ScheduledFlight`
- **Type safety** — `Flight` stores `Aircraft` instead of `Object`, so the compiler catches invalid types at compile time
- **Open/Closed Principle** — New aircraft types simply implement the `Aircraft` interface; no existing code needs modification
- **Data ownership** — Each aircraft class defines its own capacity and crew values internally, instead of having them hardcoded in `ScheduledFlight`
- **Uniform API** — All aircraft are accessed through the same interface regardless of their internal implementation
- **Reduced coupling** — `Flight` and `ScheduledFlight` depend on the `Aircraft` abstraction, not on three concrete classes

Drawbacks of Applying the Adapter Pattern

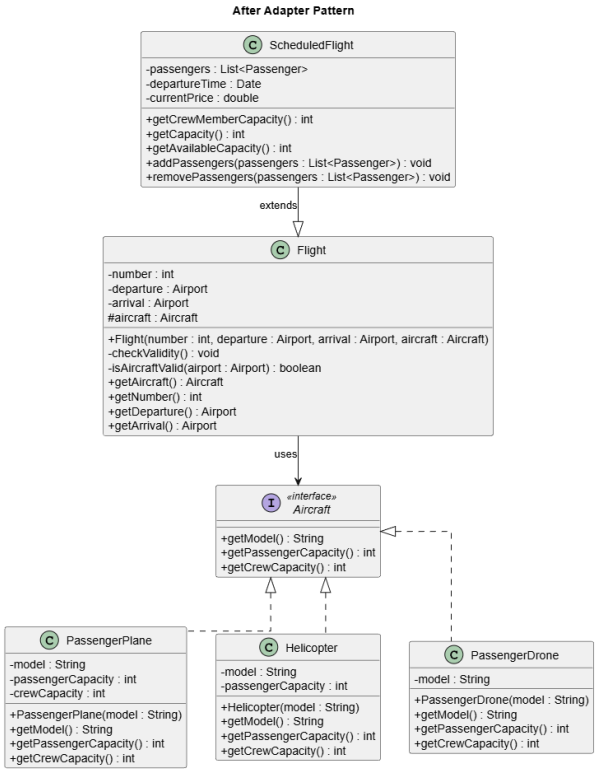
- **Additional classes/interfaces** — Introduces a new `Aircraft` interface and potentially adapter wrapper classes, increasing the number of files
- **Wrapper indirection** — If using adapter wrappers (rather than modifying originals), each call goes through an extra layer of delegation
- **Forced interface compliance** — All aircraft types must implement all methods, even if a concept doesn't naturally apply (e.g., `getCrewCapacity()` for a drone — though returning `0` is reasonable)
- **Requires modifying or wrapping existing classes** — Existing concrete classes need changes or new wrappers, which may require updating existing tests
- **Initial refactoring effort** — All usages of the raw `Object` type and `instanceof` checks must be updated simultaneously to maintain consistency

Class Diagrams

Before Adapter Pattern



Afetr Adapter Pattern



Implementation Report

The Solution: A Common Interface

We created the `Aircraft` interface to unify all aircraft types under a single contract. This interface defines three methods that every aircraft must provide: `getModel()`, `getPassengerCapacity()`, and `getCrewCapacity()`.

```java
package flight.reservation.plane;

/**
 * Common interface for all aircraft types in the reservation system.
 * This interface provides a unified way to access aircraft properties,
 * eliminating the need for instanceof checks and type casting.
 */
public interface Aircraft {

    /**
     * @return the model name of the aircraft
     */
    String getModel();

    /**
     * @return the passenger capacity of the aircraft
     */
    int getPassengerCapacity();

    /**
     * @return the crew capacity of the aircraft
     */
    int getCrewCapacity();
}
```

With this in place, all three aircraft classes— `PassengerPlane`, `Helicopter`, and `PassengerDrone` —now implement this interface. Each class adapts its unique internal structure (public fields, partial methods, or missing data) to conform to the common API.

Key Code Changes

Change 1: Eliminating instanceof Chains in `Flight.java`

The most visible improvement is how we retrieve the aircraft model. Before the refactor, `Flight.isAircraftValid()` had to check the type of aircraft and handle each one differently—13 lines of branching logic with hardcoded values. After applying the adapter pattern, this collapsed into a single polymorphic method call.

Before

```java
    private boolean isAircraftValid(Airport airport) {
        return Arrays.stream(airport.getAllowedAircrafts()).anyMatch(x -> {
            String model;
            if (this.aircraft instanceof PassengerPlane) {
                model = ((PassengerPlane) this.aircraft).model;
            } else if (this.aircraft instanceof Helicopter) {
                model = ((Helicopter) this.aircraft).getModel();
            } else if (this.aircraft instanceof PassengerDrone) {
                model = "HypaHype";
            } else {
                throw new IllegalArgumentException(String.format("Aircraft is not recognized"));
            }
            return x.equals(model);
        });
    }
```

After

```java
private boolean isAircraftValid(Airport airport) {
    return Arrays.stream(airport.getAllowedAircrafts()).anyMatch(x -> x.equals(this.aircraft.getModel()));
}
```

The method doesn't care what type of aircraft it's dealing with—it just calls `getModel()` and lets each class handle it appropriately.

Change 2: Simplifying Capacity Checks in `ScheduledFlight.java`

Previously, getting the passenger capacity required checking each aircraft type and either accessing a field, calling a method, or returning a hardcoded value. The crew capacity logic was even messier, with values like `2` for helicopters and `0` for drones scattered throughout the consumer code. Both methods had to throw `NoSuchFieldException` to handle unknown aircraft types.

Before

```java
public int getCapacity() throws NoSuchFieldException {
    if (this.aircraft instanceof PassengerPlane) {
        return ((PassengerPlane) this.aircraft).passengerCapacity;
    }
    if (this.aircraft instanceof Helicopter) {
        return ((Helicopter) this.aircraft).getPassengerCapacity();
    }
    if (this.aircraft instanceof PassengerDrone) {
        return 4;
    }
    throw new NoSuchFieldException("this aircraft has no information about its capacity");
}
```

```java
public int getCrewMemberCapacity() throws NoSuchFieldException {
    if (this.aircraft instanceof PassengerPlane) {
        return ((PassengerPlane) this.aircraft).crewCapacity;
    }
    if (this.aircraft instanceof Helicopter) {
        return 2;
    }
    if (this.aircraft instanceof PassengerDrone) {
        return 0;
    }
    throw new NoSuchFieldException("this aircraft has no information about its crew capacity");
}
```

After

```java
public int getCapacity() {
    return this.aircraft.getPassengerCapacity();
}
public int getCrewMemberCapacity() {
    return this.aircraft.getCrewCapacity();
}
```

Both methods went from 11 lines with error handling to just 3 lines of straightforward polymorphic calls. The hardcoded values are now encapsulated inside each aircraft class where they belong. As a bonus, we eliminated the `NoSuchFieldException` entirely since the interface guarantees every aircraft has these methods.

Change 3: Ripple Effect - Exception Handling Removed in `Customer.java`

Since `getAvailableCapacity()` no longer throws exceptions, all consuming code became simpler. The `Customer` class previously needed try-catch blocks when validating orders.

Before

```java
private boolean isOrderValid(List<String> passengerNames, List<ScheduledFlight> flights) {
    boolean valid = true;
    valid = valid && !FlightOrder.getNoFlyList().contains(this.getName());
    valid = valid && passengerNames.stream().noneMatch(passenger -> FlightOrder.getNoFlyList().contains(passenger));
    valid = valid && flights.stream().allMatch(scheduledFlight -> {
        try {
            return scheduledFlight.getAvailableCapacity() >= passengerNames.size();
        } catch (NoSuchFieldException e) {
            e.printStackTrace();
            return false;
        }
    });
    return valid;
}
```

After

```java
private boolean isOrderValid(List<String> passengerNames, List<ScheduledFlight> flights) {
    boolean valid = true;
    valid = valid && !FlightOrder.getNoFlyList().contains(this.getName());
    valid = valid && passengerNames.stream().noneMatch(passenger -> FlightOrder.getNoFlyList
    ().contains(passenger));
    valid = valid && flights.stream().allMatch(scheduledFlight ->
        scheduledFlight.getAvailableCapacity() >= passengerNames.size()
    );
    return valid;
}
```

The try-catch block disappeared because the interface guarantees all aircraft have capacity methods.

Similar simplifications occurred in `FlightOrder.java` and anywhere else that called aircraft capacity methods.

How Each Aircraft Adapts

Each of the three aircraft classes had to adapt differently to fit the interface:

- **PassengerPlane** already had all the data as public fields. It just needed to add getter methods that return those fields.
- **Helicopter** had some of the interface already (it had `getModel()` and `getPassengerCapacity()`), but was missing crew capacity. We added `getCrewCapacity()` returning `2`, bringing the hardcoded value from `ScheduledFlight` into the class itself.
- **PassengerDrone** was the most incomplete—it only stored the model internally with no public accessors at all. We implemented all three interface methods, returning `"HypaHype"` for the model, `4` for passenger capacity, and `0` for crew capacity. Again, these values were previously hardcoded in consumer classes.